

Response to Reviewers

Manuscript JSA-D-26-01708 (transferred to the Journal of Parallel and Distributed Computing)
CAMP: Graph-Aware Prefetching and Pinning for LLM Inference over Host-Mediated CXL Memory
(previous title: *CAMP: Content-Aware Memory Prefetching for High-Performance CXL-Based Inference*)

Quang-Vinh Dang, British University Vietnam — 5 October 2026

We thank both reviewers for detailed and constructive reports. Their main concerns were (i) an implausible hardware path, (ii) an emulator whose curves looked synthetic, (iii) an algorithmic contribution that reduced to frequency-based pinning, and (iv) an evaluation on stylised workloads and a tiny model. We agree with the substance of these points. Rather than patching the old evaluation we rebuilt it: the manuscript now has a new system model, a new simulator validated against a real GPU, a reformulated pinning algorithm, and a new evaluation on Llama-2/3 and Gemma-2 configurations with prefill, decode and mixed iterations. Section and figure numbers below refer to the revised manuscript. Where the revision does not fully resolve a point, we say so.

Summary of what changed.

- Related work reduced from about five pages to about one (Section 2); PIM, distributed training and fault-tolerant DSM material removed.
- New host-mediated GPU-CXL path model (Section 3.1, Fig. 1, Table 2); the GPU does not address the Type-3 device.
- New event-driven simulator with roofline compute, measured-data-anchored link parameters and noise; its pipeline logic is validated on a real GPU (Section 5.2, Table 4, Fig. 2).
- Algorithm 2 replaced by stall-cost-aware pinning (SCP) with an exact problem statement (Eqs. 6–7); frequency/LFU/hot-cold/FlexGen-style baselines and an exhaustive-optimum comparison added (Sections 6.3–6.4).
- Compute-time estimate obtained at runtime and its sensitivity quantified (Section 4.1, Section 6.5).
- Workloads: prefill/decode/mixed iterations on 7B–405B configurations; distilgpt2 and the stylised scenarios removed (Sections 5.3, 6.1–6.2, 6.6).

Reviewer 1

R1.1 — Section 2 is bloated and partly irrelevant.

Section 2 spans over 5 pages and reads like a generic survey; PIM, distributed training and fault-tolerant DSM have no relevance to weight prefetching.

Response. Agreed. Section 2 is now about one page with three paragraphs: how a GPU reaches CXL memory, weight offloading for LLM inference, and prefetching under a known access order, plus a positioning table (Table 1). PIM/near-data computing, distributed training and serving, fault-tolerant DSM, and the generic characterisation material were removed. KV-cache offloading and prefill/decode disaggregation are mentioned in two sentences as complementary.

R1.2 — Hardware architecture (Fig. 1): GPUs cannot be directly attached to CXL Type-3 devices.

Type-3 devices are CXL.mem endpoints mapped into the host CPU’s memory space; current GPUs cannot control them without host-mediated PCIe transactions or CXL switches.

Response. The reviewer is right and the old figure was misleading. Fig. 1 is redrawn and Section 3.1 states the path explicitly: the expander is a CPU-less NUMA node; a host runtime thread page-locks the CXL-backed region and issues `cudaMemcpyAsync` on a copy stream; the GPU copy engine reads through the root complex; kernels wait on CUDA events. The effective bandwidth is bounded by the slower of the CXL device and the PCIe link (Eq. 1). We cite measurements showing exactly this behaviour on real devices (a plain copy goes through a bounce buffer; page-locking the region enables direct DMA; 10–18 GB/s achieved) and model the bounce-buffer (“staged”) variant. H100 appears only as the compute model of the GPU. GPU-initiated direct access (NVLink-C2C, TMA-based) and CXL switches are different paths and are listed as not modelled (Sections 2, 3.1, 7).

R1.3 — Limited novelty; similar to PowerInfer.

Algorithm 1 only issues async prefetches while cumulative transfer fits the compute duration; Algorithm 2 is an offline greedy fractional knapsack on static counts, similar to PowerInfer’s hot/cold partition.

Response. We agree that the original algorithms were thin, and we no longer claim a novel frequency-based partition. (a) The prefetcher is now a work-conserving controller with next-use admission (Algorithm 1, Section 4.2); we show that the old compute-gated rule never binds in our workloads and gives identical results (Section 6.5). (b) Pinning is now SCP (Section 4.3): its objective is the predicted iteration time of a pipeline planner, solved by plan-and-verify (candidate sets, local search and annealing, dry-run replay). (c) We implemented FlexGen-style (prefix residency) and PowerInfer-style (frequency hot/cold) baselines in the same simulator and report where they are as good as CAMP: in link-bound decode, Hot/Cold is only 3.6–4.8% slower than CAMP (Section 6.6, Fig. 10a), so frequency partitioning is a sound choice there. (d) The positioning is stated in Section 2 and Table 1. What we claim instead is a regime analysis (Eq. 3, Fig. 3), the finding that placement of pinned layers decides performance when compute and transfer are comparable (frequency pinning leaves the link 12–21% idle and is 1.23–1.40× slower than SCP in the balanced 18 GB/s prefill cases, Fig. 5), and a validated simulator. We acknowledge that the contribution is an engineering-level policy and analysis rather than a new algorithmic principle.

R1.4 — Algorithm 2’s reuse formulation degenerates without tied weights.

$F(v)$ is uniform across regular layers; the algorithm degenerates to arbitrary static ordering and should consider true reuse distance or DAG depth.

Response. Agreed, and the new text says so explicitly. (a) SCP does not rank layers by $F(v)$; it minimises the predicted iteration time over pin sets, which depends on position in the sequence, layer sizes and the overlap of compute with transfer (Eqs. 6–9). (b) Transient layers are evicted by true next-use distance (Section 4.3, Algorithm 1). (c) We evaluate three workloads where $F(v)$ genuinely differs (tied embedding, speculative decoding, encoder–decoder; Table 8, Fig. 6). Frequency matters a lot for the encoder–decoder (frequency pins 2.7× faster than LRU) and little for speculative decoding, and SCP gains 1.3× over frequency in the tied-embedding case because it pins the large tied table. (d) For uniform dense models the frequency rule degenerates exactly as the reviewer says; there the benefit of SCP comes from placement (Section 6.3), not frequency.

R1.5 — Stage definition in Section 4.2 is unrealistic.

Thinking/streaming/thrashing scenarios do not map onto prefill and decode.

Response. The three scenarios were removed. Workloads are now prefill passes, decode steps and chunked-prefill mixed iterations (Section 3.2, 5.3), built from real model configurations with roofline compute (Eq. 2, Table 3). The regime map (Fig. 3) shows how the tokens per iteration, which is what distinguishes these phases, determine whether streaming is viable.

R1.6 — Emulation accuracy: curves look synthetic (linear in hit ratio, in depth; abrupt peak).

Fig. 4 is linear in hit ratio, Fig. 8(a) perfectly linear in depth, Fig. 8(c) has an abrupt peak; these weaken the accuracy of the emulation.

Response. The old curves were products of uniform synthetic layers and a single-server queue, and we have replaced that evaluation. In the new simulator, compute follows a roofline model for each layer type and batch descriptor, the link has a per-copy fixed cost, chunking, noise and slow bandwidth drift, and cache effects arise from real layer sizes and the KV-cache-limited cache capacity. Results are no longer smooth: latency versus model size has a knee where the model stops fitting (Fig. 9), and the cache-share sweep is non-monotonic where the pipeline planner and the actual overlap disagree (Fig. 10c; we report the worst points, 8.3% above the lower bound). The conclusions are stable under an $8\times$ increase of the noise level (Section 6.7, Fig. 11a). Most importantly, the simulator’s pipeline logic is compared with a real GPU (R1.7.4).

R1.7.1 — distilgpt2 (312 MB) is insufficient. Response. Agreed; distilgpt2 was removed. The evaluation uses Llama-2-7B/13B, Llama-3-8B/70B/405B and Gemma-2-9B configurations with public hyper-parameters (Table 3); the 70B model (141 GB) is evaluated with the HBM that remains after its KV cache (39% cached), and Fig. 9 spans 7B to 405B. We do not run these models’ weights; the simulator tracks bytes and times, which is stated in Section 5.1.

R1.7.2 — Graph-aware pinning ties with LFU; does CAMP offer zero algorithmic gain?

Response. The tie was an artefact of the only layer with a frequency difference (the tied embedding). Table 7 now isolates eviction on a uniform decode loop: LRU, FIFO and LFU are identical with zero hits (LFU has no signal on a cyclic scan), random eviction gets 7%, next-use eviction 32.5%. Table 8 and Section 6.4 show when frequency information helps (encoder-decoder, speculative decoding, tied embeddings) and when SCP adds to it. We state plainly that in link-bound decode the gain of SCP over frequency pinning is about 3%.

R1.7.3 — Batch scaling: 257.3 to 263.0 tokens/s indicates a miscalibrated compute model.

Response. Agreed; that experiment was removed. In the new model weight traffic is independent of batch size and compute follows the roofline, so throughput scales with the batch while the iteration is link-bound: decode throughput grows from 1.1 to 583 tokens/s for $B = 1 \rightarrow 512$ (13B, 18 GB/s), and the slowdown relative to resident weights falls from $79\times$ to $9.0\times$; resident weights saturate at about 5 200 tokens/s (Section 6.6, Fig. 8).

R1.7.4 — No cycle-accurate simulator or hardware counters. Response. We did not use gem5: no cycle-accurate model of an H100 with a CXL Type-3 device exists at the scale needed, and CXL-DMSim-style simulators do not model GPU kernels (Section 7). Instead we validated the simulator’s novel part, the copy path and the overlap logic, against hardware: a CUDA streaming loop on a Tesla T4 (pinned host memory standing in for the CXL pool) with measured compute-only and copy-only components, composed in the simulator without fitting and compared with 30 measured composite configurations: 7.6% mean absolute error, 4.0% in link-bound configurations, up to 25% where compute and copy are comparable because of GPU clock/power effects the simulator

does not model (Section 5.2, Table 4, Fig. 2). The measured pageable/pinned bandwidth ratio (0.37) is consistent with the bounce-buffer penalty reported for CXL regions. We also check the engine against closed-form pipeline times and the pin planner against the engine (0.18% mean error). The remaining limitation is that no CXL hardware, PCIe Gen5 GPU or H100 was measured; this is stated in Sections 1 and 7.

Reviewer 2

R2.1 — Eq. (2) is a 0–1 knapsack but Algorithm 2 sorts by reuse frequency. Response. The reviewer is correct that the two were not equivalent. Section 4.3 now separates three objects: (i) the SCP problem (Eq. 6), minimising predicted iteration time subject to a capacity constraint that includes the streaming slot; this is not a knapsack because the objective is not additive and depends on placement; (ii) the knapsack proxy (Eq. 7), whose benefit per unit size is $F(u)t_{\text{copy}}(s_u)/s_u \approx F(u)/BW_{\text{eff}}$, solved exactly by dynamic programming and used as a baseline; (iii) the old frequency-ordered greedy, which is the special case for uniform sizes. We compare all three, plus random, prefix and stride sets (Fig. 5) and, on an 18-unit model, the exhaustive optimum: SCP is 0.7% above it on average (maximum 4.2%) against 21.1% for frequency ranking (Table 6). SCP remains a heuristic without a guarantee.

R2.2 — $T_{\text{comp}}(v_i)$ is required but the paper claims no profiling. Response. Section 4.1 now answers this. $\hat{t}_u$ is computed from tensor shapes and the live batch descriptor with a roofline using generic device-class constants and the peak rates reported by the driver, and is calibrated online by an exponential moving average of observed/nominal duration per layer kind (Eq. 5). The layer’s own duration is not needed before it completes. This removes offline profiling of the model or workload but still needs the first iteration (or a probe iteration) to calibrate. Sensitivity is quantified in Section 6.5 (Table 9, Fig. 7): the generic estimate has 23% error for prefill and one probe iteration reduces it to 0.5–2.7%; prefetching is insensitive to the estimate (identical latencies for biases from $0.25\times$ to $4\times$); pinning is sensitive in the balanced regime when uncalibrated (up to 24% loss at bias 2–4 for 70B) and insensitive after calibration (below 3%).

R2.3 — Evidence for Graph-Aware Pinning is weak; it ties with LFU. Response. See R1.4 and R1.7.2. In addition, the revised evaluation shows where the pin rule matters: contiguous (prefix or frequency-ranked) pin sets leave the link idle (79–88% utilisation versus 99.5–100% for interleaved sets) and are $1.23\text{--}1.40\times$ slower than SCP when compute and transfer are comparable, and even random pinning beats frequency ranking by 13–25% there (Section 6.3, Fig. 5, Eq. 10). We also report where there is no difference (link-bound decode, compute-bound large prefill).

R2.4 — The CXL execution path in the emulator needs a concrete description. Response. See R1.2. Section 3.1 states who initiates the transfer (host runtime thread), how data reaches HBM (DMA from page-locked CXL-backed memory through the root complex), the synchronisation (CUDA events), and the modelled overheads (per-copy API/driver/first-byte cost, per-chunk cost, event synchronisation cost, bandwidth bound by the slower link, noise and drift, staged bounce-buffer variant); Table 2 lists the values and their basis. Not modelled: CXL switches, interleaving of several expanders (folded into the effective bandwidth, swept from 8 to 64 GB/s), GPU-initiated direct access, coherence traffic.

Other changes

- The previous multi-tenant and synchronised-burst serving experiments were removed because they depended on the old single-queue model and could not support the claims made from them.
- The abstract, introduction and conclusion were rewritten to state limitations and negative results (decode is link-bound; gains over frequency pinning are small there).
- Code, raw results and the hardware micro-benchmark are in the public repository referenced in the Data availability statement; every table and figure is generated from the result files.
- The generative-AI declaration required by the journal is included in the manuscript.